

Relazione del Primo Progetto di Big Data

Piattaforma Integrata per il Monitoraggio e l'Analisi del Traffico di Rete

Andrea Macale (560793) — A.A. 2025/2026

Il progetto realizza una **piattaforma Big Data end-to-end** per l'analisi del traffico di rete. L'infrastruttura simula un ambiente enterprise attraverso **Kathara** (container Docker) e integra **Apache Spark** per l'elaborazione distribuita di oltre 66 milioni di record, **MongoDB** come Data Lake operazionale e modelli **LLM locali** (Ollama) per l'assistente AI. Il tutto è fruibile tramite una dashboard **Streamlit** accessibile via browser.

1 Architettura dell'Infrastruttura

1.1 Topologia di Rete

La rete è simulata su Kathara e organizzata in **tre Sistemi Autonomi (AS)**. **AS1** ospita la rete locale con i client e il nodo attacker per i test di sicurezza. **AS2** è il Data Center, cuore del sistema: implementa un fabric **Clos EVPN-VXLAN** (Top-of-Fabric, Spine, Leaf) e ospita tutti i servizi applicativi. **AS3** simula la connettività Internet esterna con DNS autoritativo. Il router **r5** funge da gateway perimetrale del Data Center e ospita il firewall dinamico.

1.2 Servizi del Data Center

Nodo	Immagine custom	Ruolo
spark	kathara-spark-custom	Apache Spark Master + Worker
mongo_db	kathara-mongo-custom	MongoDB — Data Lake
llm	kathara-llm-custom	Ollama + GPU passthrough
admin	kathara-admin-custom	Dashboard Streamlit
web_server	kathara-webserver-custom	Web server statico
sniffer	kathara/base	Cattura traffico (tcpdump)

Tabella 1: Nodi del Data Center e rispettivi ruoli

2 Il Dataset: BigFlow-NIDS

2.1 Origine e Caratteristiche

Il dataset **BigFlow-NIDS** è disponibile su Kaggle e raccoglie flussi di traffico etichettati per il rilevamento delle intrusioni di rete (NIDS). Contiene oltre **66.355.798 record** suddivisi tra traffico legittimo (Benign) e diverse categorie di attacco (DDoS, Brute Force, ecc.).

Il dataset viene convertito da CSV a **Apache Parquet** tramite Spark al primo avvio (Listato ??). Parquet garantisce compressione columnar, schema-on-read e compatibilità nativa con Spark SQL senza parsing aggiuntivo.

```
1 df = spark.read \
2     .option("header", "true") \
3     .option("inferSchema", "true") \
4     .option("samplingRatio", "0.1") \
5     .csv("/opt/spark/data/raw/BigFlow-NIDS.csv")
6 df = df.withColumn("label", df["label"].cast("string"))
7 df.write.mode("overwrite") \
8     .parquet("/opt/spark/data/processed/BigFlow-NIDS.parquet")
```

Listato 1: Ingestione CSV e conversione in Parquet

2.2 Schema Principale

I campi chiave utilizzati nelle query analitiche sono riportati in Tabella ??.

Campo	Tipo	Descrizione
IPV4_SRC_ADDR	String	IP sorgente del flusso
IPV4_DST_ADDR	String	IP destinazione del flusso
L4_SRC_PORT / L4_DST_PORT	Integer	Porta sorgente / destinazione
PROTOCOL	Integer	Protocollo (6=TCP, 17=UDP, 1=ICMP)
IN_BYTES / OUT_BYTES	Long	Byte ricevuti / trasmessi
IN_PKTS / OUT_PKTS	Long	Pacchetti ricevuti / trasmessi
FLOW_DURATION_MILLISECONDS	Long	Durata del flusso in ms
FLOW_START_MILLISECONDS	Long	Timestamp di inizio (epoch ms)
label / Attack	String	Benign / tipo di attacco

Tabella 2: Schema del dataset BigFlow-NIDS

3 MongoDB come Data Lake

3.1 Motivazione della Scelta

MongoDB svolge il ruolo di **Data Lake operativo** per i dati live e semi-strutturati, complementare al Parquet storico. I motivi principali della scelta sono:

- **Schema flessibile:** documenti a struttura variabile, ideali per stream PCAP e alert;
- **Capped Collections:** buffer circolari nativi a dimensione fissa per dati live;
- **Connettore Spark:** mongo-spark-connector_2.12-10.3.0.jar per Data Federation;
- **Ricerca full-text:** filtri \$regex per il Data Catalog.

3.2 Struttura del Database dataLake

Il database dataLake contiene otto collezioni:

Collezione	Tipo	Contenuto
live_traffic	Capped (5.000 doc, 10 MB)	Pacchetti live dal sniffer PCAP
alerts	Standard	Alert generati dall'ingestor real-time
blocked_ips	Standard	IP bloccati con timestamp e motivo
black_list	Standard	IP in blocco automatico immediato
white_list	Standard	IP sempre considerati sicuri
metadata_catalog	Standard	Data Catalog dei dataset disponibili
llm_models	Standard	Configurazione modelli LLM
audit_logs	Standard	Log di ogni azione sulla dashboard

Tabella 3: Collezioni del database dataLake

3.3 Capped Collection per il Traffico Live

La collezione live_traffic è di tipo **Capped**: struttura circolare in cui i documenti più vecchi vengono sovrascritti automaticamente al raggiungimento del limite configurato, mantenendo costante la dimensione del database:

```

1 db.create_collection(
2   "live_traffic",
3   capped=True,
4   size=10 * 1024 * 1024, # 10 MB
5   max=5000               # max 5000 documenti
6 )

```

Listato 2: Creazione della Capped Collection

Ogni documento inserito ha struttura: timestamp, summary (descrizione Scapy), length, src, dst, proto.

4 Pipeline di Ingestione Dati

4.1 Ingestor PCAP in Tempo Reale

Il modulo `pcap_ingestor.py` monitora in polling (ogni secondo) il file `analisi_traffico.pcap` prodotto dal nodo sniffer. Il suo comportamento è descritto nell'Algoritmo ??.

Algoritmo 1 PCAP Ingestor

```

1: ultimo ← 0
2: while True do
3:   pacchetti ← LEGGIPCAP(pcap_path)
4:   if |pacchetti| < ultimo then
5:     ultimo ← 0
6:   end if
7:   if |pacchetti| > ultimo then
8:     nuovi ← pacchetti[ultimo :]
9:     for ogni pacchetto p in nuovi do
10:      Estrai src, dst, proto, summary
11:      if src ∈ black_list and src ∉ blocked then
12:        Invia BLOCK:src via UDP al firewall su r5
13:        Inserisci in blocked_ips
14:      end if
15:      Analizza pattern TCP (BGP, FTP, SSH, HTTP, MongoDB)
16:      if pattern rilevato and src ∉ white_list then
17:        Inserisci alert in alerts (CRITICAL/INFO)
18:      end if
19:    end for
20:    Inserisci nuovi in live_traffic
21:    ultimo ← |pacchetti|
22:  end if
23:  ATTENDI(1 s)
24: end while

```

▷ Rilevata rotazione del file

4.2 Data Federation Spark + MongoDB

Il Data Lake Explorer monta due View federate interrogabili con una singola query SQL: `storico_parquet` (file Parquet) e `live_mongo` (collezione MongoDB). Esempio di join cross-source:

```

1 SELECT s.IPV4_SRC_ADDR, l.summary
2 FROM storico_parquet s
3 JOIN live_mongo l ON s.IPV4_SRC_ADDR = l.src
4 LIMIT 10

```

Listato 3: Esempio di query federata Parquet + MongoDB

5 Analisi del Traffico: Query Spark SQL

Tutte le analisi vengono eseguite via Spark SQL su oltre 66 milioni di record. I file `.sql` risiedono in `storage/src/analytics/` e vengono caricati dinamicamente dalla dashboard, separando la logica applicativa dalle interrogazioni dati.

5.1 Classificazione degli Attacchi per Vettore

La query `rilevamento_anomalie.sql` classifica i flussi malevoli usando una logica rule-based su porte di destinazione e volumi di traffico, espressa come CTE (Listato ??).

```

1 WITH traffico AS (
2   SELECT
3     CASE
4       WHEN l4_dst_port = 179 OR l4_src_port = 179
5       THEN 'BGP Hijacking/Exploit'

```

```

6      WHEN l4_dst_port = 22 THEN 'SSH Brute Force'
7      WHEN l4_dst_port = 53 THEN 'DNS Amplification'
8      WHEN l4_dst_port IN (80, 443)
9          THEN 'Web Attack (HTTP/S)'
10     WHEN l4_dst_port = 21 THEN 'FTP Exploit'
11     WHEN flow_duration_milliseconds < 100
12         AND in_pkts > 1000 THEN 'L3/L4 Flood (DDoS)'
13     WHEN in_pkts > 5000 AND in_bytes > 100000
14         THEN 'Volumetric DDoS'
15     ELSE 'Altro / Malicious General'
16 END AS vettore_attacco,
17 label AS etichetta_dataset,
18 COUNT(*) AS occorrenze,
19 ROUND(AVG(flow_duration_milliseconds), 2)
20     AS durata_media_flusso,
21 ROUND(SUM(in_pkts + out_pkts), 0) AS pacchetti_totali,
22 SUM(in_bytes + out_bytes) AS traffico_b
23 FROM traffico_nids
24 WHERE label != 'Benign'
25 GROUP BY 1, 2
26 )
27 SELECT vettore_attacco, etichetta_dataset, occorrenze,
28        durata_media_flusso, pacchetti_totali, traffico_b,
29        CASE
30            WHEN traffico_b >= 1099511627776
31                THEN CONCAT(ROUND(traffico_b/1099511627776.0,2), 'TB')
32            WHEN traffico_b >= 1073741824
33                THEN CONCAT(ROUND(traffico_b/1073741824.0,2), 'GB')
34            WHEN traffico_b >= 1048576
35                THEN CONCAT(ROUND(traffico_b/1048576.0,2), 'MB')
36            WHEN traffico_b >= 1024
37                THEN CONCAT(ROUND(traffico_b/1024.0,2), 'KB')
38            ELSE CONCAT(traffico_b, 'B')
39        END AS traffico_h
40 FROM traffico
41 ORDER BY traffico_b DESC

```

Listato 4: Classificazione rule-based dei flussi anomali

La colonna `traffico_h` converte automaticamente i byte in formato leggibile (KB, MB, GB, TB) tramite soglie potenze di 1024. Il grafico risultante (Figura ??) mostra i vettori d'attacco ordinati per volume di traffico totale generato.

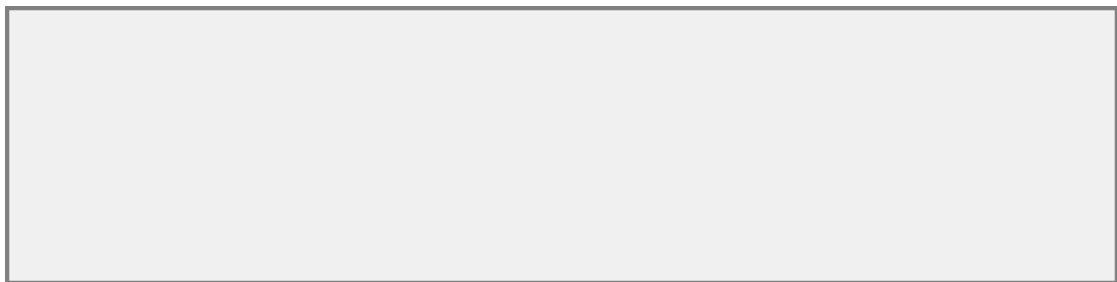


Figura 1: Classificazione dei flussi anomali per vettore d'attacco (barre orizzontali Altair)

5.2 Top 5 Attaccanti

La query `top_attaccanti.sql` identifica i cinque IP sorgente responsabili del maggior numero di flussi malevoli:

```

1 SELECT ipv4_src_addr AS ip_attaccante,
2        COUNT(*)      AS occorrenze,
3        SUM(in_bytes + out_bytes) AS traffico_b,
4        label AS classe
5 FROM traffico_nids
6 WHERE label != 'Benign'
7 GROUP BY ipv4_src_addr, label
8 ORDER BY occorrenze DESC
9 LIMIT 5

```

Listato 5: Top 5 IP attaccanti per numero di flussi

Dalla dashboard, un click sull'IP nella tabella risultante trasmette via UDP il comando BLOCK:<ip> al demone firewall su r5.

5.3 Bilanciamento del Dataset

```
1 SELECT Attack AS label, COUNT(*) AS occorrenze
2 FROM traffico_nids
3 GROUP BY Attack
```

Listato 6: Distribuzione delle classi nel dataset

Il risultato (Figura ??) evidenzia l'eventuale **class imbalance**: un dataset fortemente sbilanciato verso la classe Benign porta a modelli di classificazione con alta accuracy ma bassa recall sulle classi di attacco, problematica rilevante in ambito NIDS.

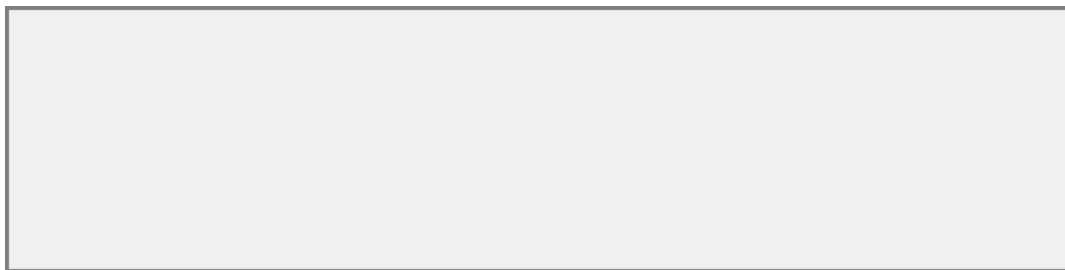


Figura 2: Distribuzione delle classi nel dataset BigFlow-NIDS

5.4 Distribuzione Protocolli e Porte

```
1 SELECT
2     L4_DST_PORT AS port,
3     CASE
4         WHEN PROTOCOL = 6 THEN 'TCP'
5         WHEN PROTOCOL = 17 THEN 'UDP'
6         WHEN PROTOCOL = 1 THEN 'ICMP'
7         ELSE CAST(PROTOCOL AS STRING)
8     END AS protocol_name,
9     label,
10    COUNT(*) AS flow_count,
11    AVG(IN_BYTES + OUT_BYTES) AS avg_bytes
12 FROM traffico_nids
13 GROUP BY 1, 2, 3
14 ORDER BY flow_count DESC
15 LIMIT 50
```

Listato 7: Distribuzione per porta di destinazione e protocollo

Il grafico risultante è uno **stacked bar chart** (Figura ??): asse Y con le 50 porte più trafficate, asse X con il numero di flussi, colore per classificazione. Permette di identificare immediatamente le porte “calde” (80, 443, 22, 53, 179).



Figura 3: Distribuzione dei flussi per porta di destinazione e protocollo (stacked bar)

5.5 Andamento Temporale del Traffico

```

1 SELECT
2     date_trunc('minute',
3         from_unixtime(FLOW_START_MILLISECONDS / 1000)
4     ) AS time_window,
5     label,
6     SUM(IN_BYTES + OUT_BYTES) AS total_bytes,
7     COUNT(*) AS flow_count
8 FROM traffico_nids
9 GROUP BY 1, 2
10 ORDER BY 1 ASC

```

Listato 8: Serie storica del traffico aggregata per minuto

`date_trunc('minute', ...)` aggrega i flussi in finestre di un minuto; `from_unixtime` converte il timestamp da millisecondi a formato datetime di Spark. Il grafico (Figura ??) è un **line chart multi-serie**, una linea per ogni label, che permette di identificare picchi e finestre di attività malevola.

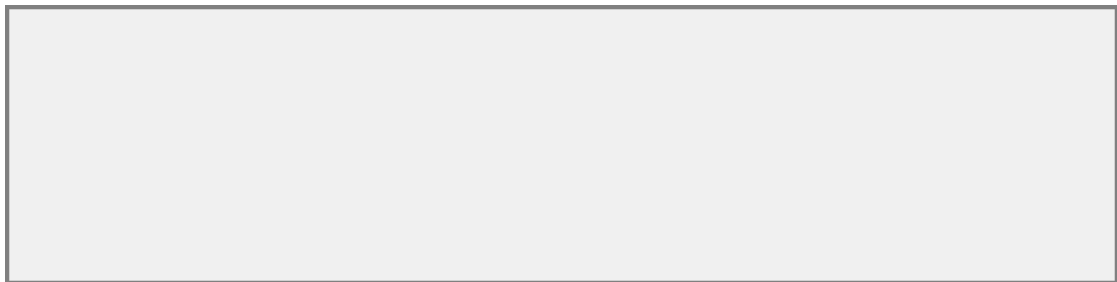


Figura 4: Andamento temporale del traffico per categoria (line chart Altair)

5.6 Top Talkers — Matrice Sorgente/Destinazione

```

1 SELECT
2     IPV4_SRC_ADDR AS source_ip,
3     IPV4_DST_ADDR AS destination_ip,
4     label,
5     SUM(IN_BYTES + OUT_BYTES) AS total_bytes,
6     COUNT(*) AS flow_count
7 FROM traffico_nids
8 WHERE IPV4_SRC_ADDR IS NOT NULL
9     AND IPV4_DST_ADDR IS NOT NULL
10 GROUP BY 1, 2, 3
11 ORDER BY total_bytes DESC
12 LIMIT 100

```

Listato 9: Heatmap delle coppie IP per volume di traffico

Il risultato è una **heatmap** (Altair `mark_rect`, Figura ??): matrice IP sorgente $\times$ IP destinazione, intensità del colore (scala `reds`) proporzionale al volume di byte. Particolarmente efficace per evidenziare attacchi volumetrici in cui un singolo IP sorgente appare in molte righe della matrice.

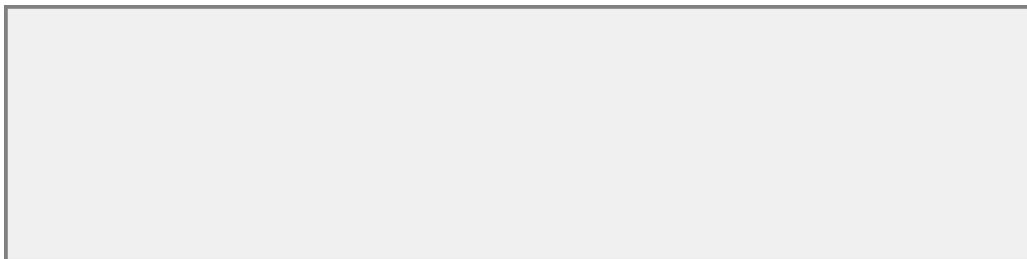


Figura 5: Heatmap delle coppie IP per volume di traffico scambiato

6 Assistente IA (AI Copilot)

6.1 Architettura LLM

Il nodo `llm` ospita un'istanza **Ollama** con GPU passthrough, esposta sulla porta 11434. I modelli sono configurati dinamicamente in MongoDB (collezione `llm_models`), permettendo di aggiungere nuovi modelli senza modificare il codice applicativo:

Nome	ID Ollama	Tipo	Caratteristica
Veloce	<code>llama3.2:3b</code>	<code>llama</code>	Risposte dirette, no reasoning
Ragionamento	<code>qwen3.5:9b</code>	<code>qwen</code>	Reasoning esplicito con tag <code><think></code>

Tabella 4: Modelli LLM configurati

6.2 Streaming e Separazione del Reasoning

La comunicazione con Ollama avviene tramite `/api/generate` in modalità **streaming** (chunked HTTP). Il modulo `llm_utils.py` separa i token di ragionamento (tag `<think>`) da quelli della risposta finale tramite espressione regolare, visualizzandoli in aree distinte della UI:

```

1 t_match = re.search(
2     r'<think>(.*?)(</think>|$)', raw_stream, re.DOTALL)
3 if t_match:
4     thinking_text = t_match.group(1)
5 clean_answer = re.sub(
6     r'<think>.*?(</think>|$)', '', raw_stream, flags=re.DOTALL)

```

Listato 10: Separazione reasoning / risposta finale

6.3 RAG Contestuale

Per le domande riguardanti la sicurezza, l'AI Copilot implementa una forma semplificata di **Retrieval-Augmented Generation**: prima di invocare il modello, il sistema recupera da MongoDB gli ultimi 5 IP bloccati e i 5 alert più recenti, iniettandoli nel prompt come contesto reale del Data Center. L'algoritmo di selezione è descritto nell'Algoritmo ??.

Algoritmo 2 RAG contestuale per l'AI Copilot

```

1:  $kw \leftarrow ["ip", "bloccat", "firewall", "traffico", \dots]$ 
2:  $greetings \leftarrow ["ciao", "hello", "chi sei", \dots]$ 
3:  $generic \leftarrow (len(prompt) \leq 15 \text{ or } prompt \in greetings)$ 
4: if  $generic$  then
5:      $prompt\_finale \leftarrow \text{"Rispondi in italiano: " + prompt}$ 
6: else
7:      $blocked \leftarrow \text{MongoDB.BLOCKED\_IPS.FIND}(\{"status": "BLOCKED"\}, limit = 5)$ 
8:      $alerts \leftarrow \text{MongoDB.ALERTS.FIND}(\{\}, sort = \{ts: -1\}, limit = 5)$ 
9:      $contesto \leftarrow \text{FORMATTA}(blocked, alerts)$ 
10:     $prompt\_finale \leftarrow contesto + \text{"\n Domanda: " + prompt}$ 
11: end if
12: return  $\text{OLLAMASTREAM}(model, prompt\_finale, system)$ 

```

6.4 Generazione SQL via LLM (NL2SQL)

Il **Data Lake Explorer** integra un assistente specializzato nella generazione di query Spark SQL. Il prompt di sistema (`sql_qwen.txt`) istruisce il modello a produrre *esclusivamente* codice SQL (senza markdown, senza testo extra), specificando le due tabelle disponibili (`storico_parquet`, `live_mongo`) e le rispettive colonne. La risposta viene renderizzata con syntax highlighting SQL.

7 Sistema di Sicurezza e Risposta Automatica

Il sistema implementa un **ciclo chiuso di rilevamento e risposta** illustrato in Figura ?? . Il modulo `pcap_ingestor.py` genera alert in MongoDB; la dashboard espone il pulsante di blocco; il comando raggiunge via UDP il demone `firewall_daemon.py` su `r5` che applica regole `iptables`. Lo script `sync_firewall.py`, eseguito all'avvio, ripristina tutte le regole da MongoDB garantendo la sopravvivenza al riavvio (**state recovery**).

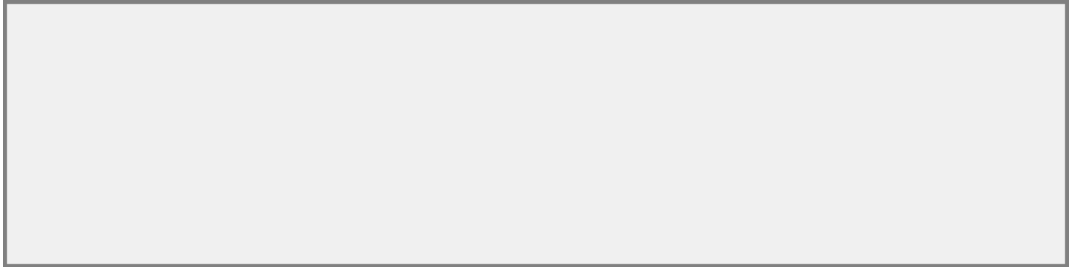


Figura 6: Ciclo chiuso di rilevamento e risposta agli incidenti

8 Dashboard Amministrativa

La dashboard Streamlit è strutturata in cinque tab principali:

Tab	Modulo	Contenuto
Homepage	<code>homepage.py</code>	Stato servizi, KPI globali, alert banner
Catalogo	<code>catalogo.py</code>	Data Catalog, anteprima, upload CSV, Data Lake Explorer
Analisi	<code>spark_analysis.py</code>	6 tab analitiche Spark SQL (cap. 5)
Sniffer	<code>live_sniffer.py</code>	Traffico live MongoDB, Privacy Mode, download PCAP
Assistente IA	<code>ai_copilot.py</code>	Chat LLM con RAG contestuale

Tabella 5: Struttura della dashboard amministrativa

Una funzionalità trasversale rilevante è il **Privacy Mode**: un toggle nella sidebar maschera l'ultimo ottetto degli indirizzi IP (es. `192.168.1.xxx`) nella vista Live Sniffer, consentendo la visualizzazione del traffico senza esporre gli indirizzi completi in ambienti condivisi. La Figura ?? mostra la homepage della dashboard.



Figura 7: Homepage della dashboard amministrativa con lo stato dei servizi

9 Conclusioni

Il progetto ha dimostrato l'applicazione pratica delle principali tecnologie Big Data in uno scenario realistico:

- **Ingestion eterogenea**: dataset storico Parquet (66M+ record) e dati live PCAP, entrambi interrogabili via Spark SQL grazie alla Data Federation.
- **Elaborazione distribuita**: Apache Spark con query SQL complesse su aggregazioni, window functions temporali e join tra sorgenti dati eterogenee.
- **Visualizzazione analitica**: sei visualizzazioni interattive Altair (barre, line chart multi-serie, heatmap) offrono prospettive complementari sul traffico di rete.

- **Architettura Data Lake:** MongoDB come Data Lake operativo (Capped Collection per stream live, Data Catalog per i metadati).
- **Integrazione AI:** LLM locali con RAG contestuale su dati MongoDB, prompt engineering e generazione automatica di query SQL (NL2SQL), senza dipendenza da servizi cloud.

Sviluppi futuri. Integrazione di un modello Spark MLlib (Random Forest o XGBoost) per la classificazione automatica del traffico; sistema di alerting proattivo basato su soglie dinamiche calcolate con Spark Structured Streaming; configurazione multi-worker Spark su nodi distinti per testare lo scaling orizzontale.